Lesson 6 Worksheet: Testing Without a Robot — ANSWER KEY

Part 1: Write the Expected Output

Test Case 1: Basic forward path - Start: (0, 0) Destination: (2, 3)

	Path
Expected:	[(1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]
Actual:	[(1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]

Expected number of steps: 5

Does expected match actual? PASS

Test Case 2: Reverse direction - Start: (2, 3) Destination: (0, 0)

	Path
Expected:	[(1, 3), (0, 3), (0, 2), (0, 1), (0, 0)]
Actual:	[(1, 3), (0, 3), (0, 2), (0, 1), (0, 0)]

Expected number of steps: 5

Does expected match actual? PASS

Test Case 3: Same row (column movement only) - Start: (1, 0) Destination: (1, 3)

	Path
Expected:	[(1, 1), (1, 2), (1, 3)]
Actual:	[(1, 1), (1, 2), (1, 3)]

Expected number of steps: 3

Does expected match actual? PASS

Test Case 4: Same position (no movement) - Start: (2, 2) Destination: (2, 2)

	Path
Expected:	[]
Actual:	[]

Expected number of steps: 0

Does expected match actual? PASS

Part 2: What Does Each Test Cover?

Test Case	What It Tests
Test 1: (0,0) to (2,3)	Both row and column movement (positive steps)
Test 2: (2,3) to (0,0)	Both row and column movement (negative steps)
Test 3: (1,0) to (1,3)	Column-only movement (same row)
Test 4: (2,2) to (2,2)	No movement (same position)

Is there a scenario that is NOT covered by these four tests? YES

If yes, what is it? Row-only movement (same column). There is no test where the start and destination share the same column but differ in row, which would test the row loop without the column loop.

Part 3: Understanding the Test Code

1. **What does the `test_name` parameter do?**

   It labels the test in the output so you know which test passed or failed.

2. **What does `actual == expected` compare?**

   It compares the two lists element by element — both lists must have the same tuples in the same order for the comparison to be `True`.

3. **When a test FAILS, what extra information is printed? Why is this helpful?**

   It prints both the expected path and the actual path. This is helpful because you can see exactly where the two paths differ, which points you toward the bug in your code.

4. **Predict the output of this call:**

```
Test A - PASS
```

5. *Predict the output of this call (with a WRONG expected value on purpose):*

   Line 1: `Test B - FAIL`

   Line 2: `Expected: [(0, 1), (1, 1)]`

   Line 3: `Actual:   [(1, 0), (1, 1)]`

   The algorithm moves rows first then columns, so the actual path goes through (1,0) then (1,1), not through (0,1).

Part 4: Design Your Own Test Cases

Your Test Case 5:

- *Description of what this tests: Row-only movement (same column)*
- *Start: (0, 2)*
- *Destination: (3, 2)*
- *Expected path: [(1, 2), (2, 2), (3, 2)]*
- *Expected number of steps: 3*

Your Test Case 6:

- *Description of what this tests: South-West direction (row increasing, col decreasing)*
- *Start: (0, 3)*
- *Destination: (2, 0)*
- *Expected path: [(1, 3), (2, 3), (2, 2), (2, 1), (2, 0)]*
- *Expected number of steps: 5*

Part 5: Checking Path Properties

Property	How to Check
Path does not include the start position	`path` should not contain `start`
Path ends at the destination	Compare `path[-1]` to `destination`
Number of steps equals Manhattan distance	`len(path)` should equal `|dest_row - start_row| + |dest_col - start_col|`

Property	How to Check
Each step moves exactly one row OR one column	For every pair of consecutive positions, the difference should be exactly 1 (either the row changes by 1 or the column changes by 1, but not both at the same time)

Part 6: Testing Checklist

- [x] Test with row increasing and col increasing (down-right)
- [x] Test with row decreasing and col decreasing (up-left)
- [x] Test with row increasing and col decreasing (down-left)
- [x] Test with row decreasing and col increasing (up-right)
- [x] Test with same row (column movement only)
- [x] Test with same column (row movement only)
- [x] Test with same position (no movement)
- [x] No path includes its start position
- [x] All paths end at the correct destination
- [x] All step counts match the Manhattan distance

How many of these tests pass? 10 / 10

Part 7: Debugging Practice

Test output:

```
Test: (3, 0) to (1, 2) - FAIL
 Expected: [(2, 0), (1, 0), (1, 1), (1, 2)]
 Actual:   [(2, 0), (1, 0), (0, 0), (-1, 0), (-2, 0)...
```

1. **What is the problem? The path keeps going instead of stopping.**

   The path passes through the correct destination row (1, 0) but keeps decrementing the row past it — going to (0, 0), (-1, 0), (-2, 0), and so on. The row loop does not stop when it reaches the destination row.

2. **Which loop is causing the problem - the row loop or the column loop?**

   The row loop. The path never switches to column movement — it just keeps changing the row value and the column stays at 0 the entire time.

3. **What mistake in the code would cause this? (Hint: think about the while condition)**

   The most likely mistake is that the while condition compares against the wrong variable — for example, `while current_row != dest_col` instead of `while current_row != dest_row`. Since `dest_col` is 2 and the row is decrementing (3, 2, 1, 0, -1, ...), `current_row` passes through 2 early on

but the loop keeps going because it already passed that value heading in the wrong direction. The row never reaches `dest_col = 2` *again, so the loop runs forever.*

Reflection

Professional software developers write tests BEFORE running code on real hardware. Why is "test on screen before you test on the floor" a good strategy for robotics?

Testing on screen first catches algorithm bugs — wrong paths, infinite loops, off-by-one errors — without risking the robot driving off the grid or crashing. It is much faster to read printed output and spot a mistake than to set up the robot, watch it drive, figure out where it went wrong, and repeat. You can run dozens of test cases in seconds on screen, but each physical robot test takes minutes. Debugging print output is also easier because you can see the exact path list and compare it to your expected answer, while watching a robot drive only gives you a rough idea of what happened.

Next Lesson: *We'll tackle the challenge of* ***turning*** *– the robot needs to face the right direction before it can drive to the next coordinate!*